Name – Earl Pratik Sinha

Roll – 2024AD05449

Email – 2024ad05449@wilp.bits-pilani.ac.in

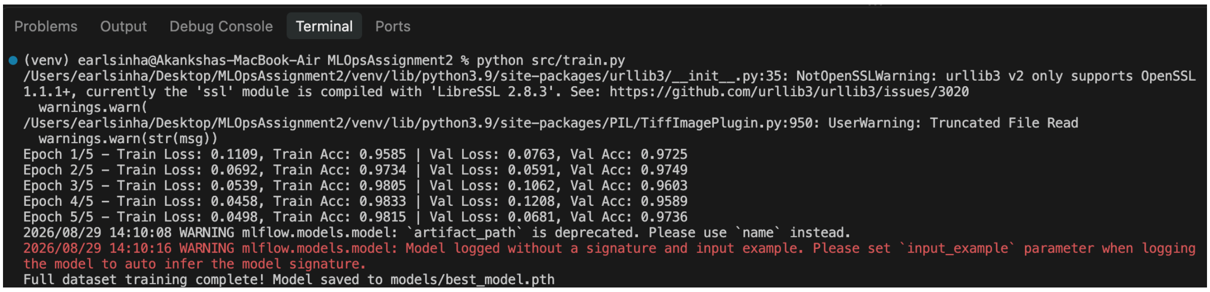
MLOps Assignment 2:

GitHub Repository: https://github.com/earlpratiksinha/MLOpsAssignment2.git

1. Model Training & Pipeline Verification

The model training was executed using PyTorch with transfer learning via MobileNetV2 over 5 training epochs.

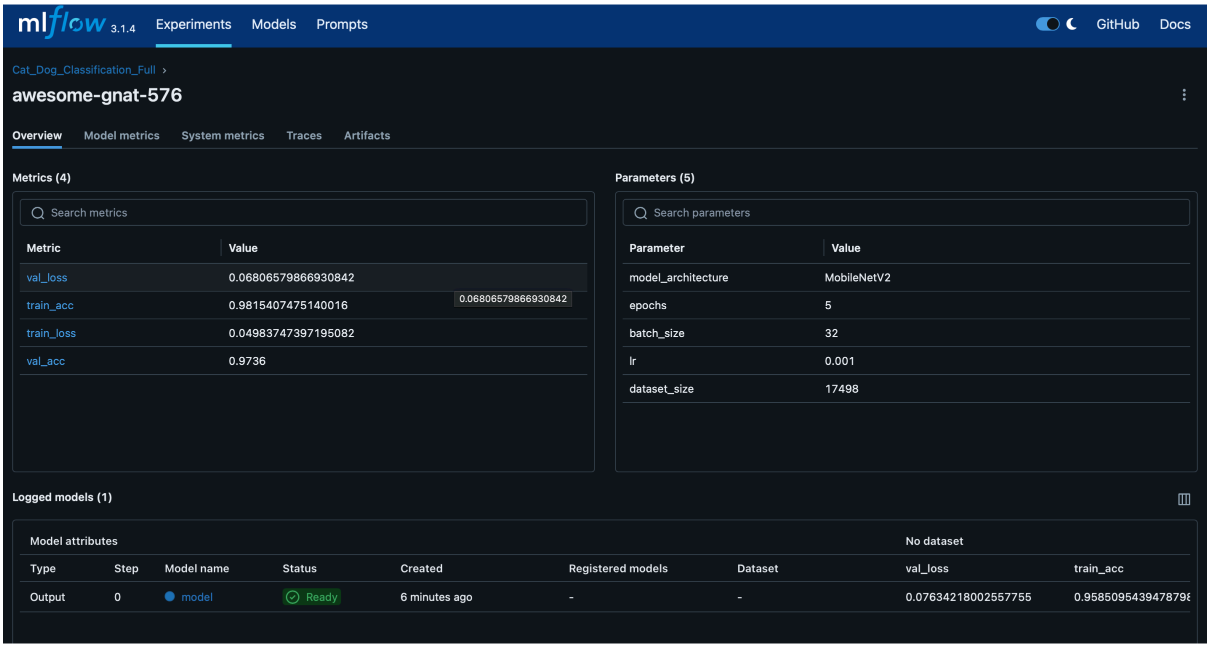
- **Training Results:** Final Training Loss reached **0.0498** with a Training Accuracy of **98.15%**. Final Validation Loss landed at **0.0681** with a Validation Accuracy of **97.36%**.
- **Artifact Storage:** The trained model state dictionary was saved to `models/best_model.pth`.



2. Experiment Tracking with MLflow

All hyperparameter configurations, evaluation metrics, and model outputs were tracked using MLflow under the experiment run `awesome-gnat-576`.

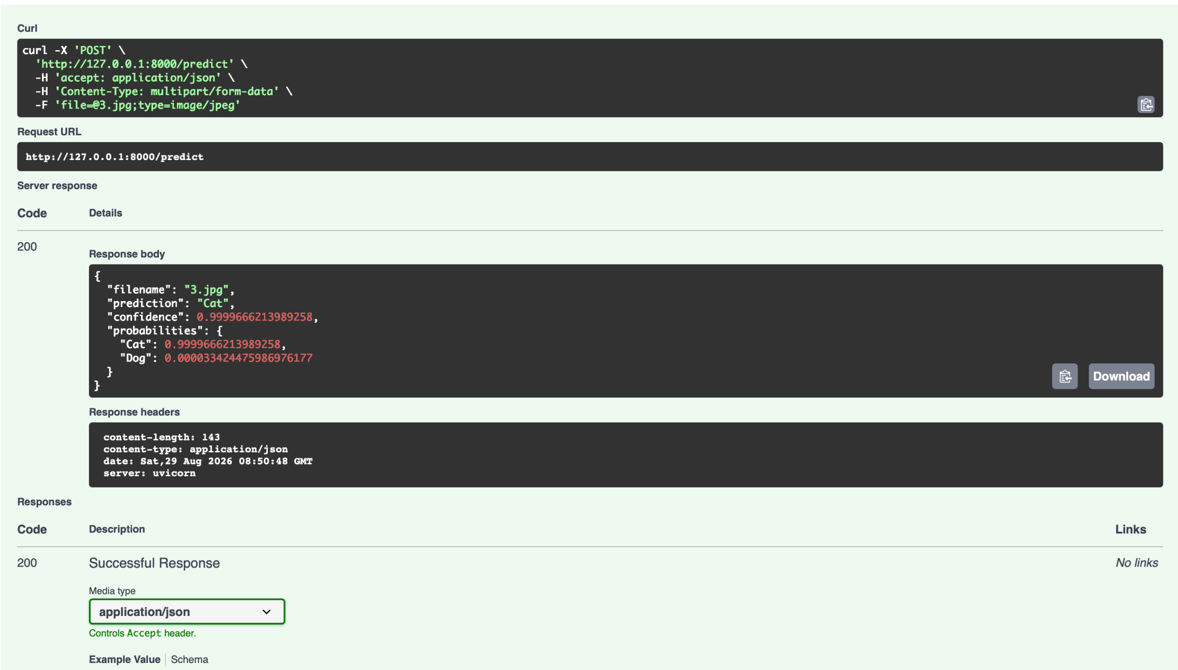
Parameter / Metric	Tracked Value
Model Architecture	MobileNetV2
Dataset Size	17,498 images
Epochs / Batch Size	5 / 32
Learning Rate (lr)	0.001
Validation Accuracy	97.36%



3. FastAPI Deployment & Real-time Inference

A RESTful API was implemented using FastAPI to serve real-time predictions. The endpoint takes an uploaded image, resizes and normalizes the input tensor, and returns prediction probabilities.

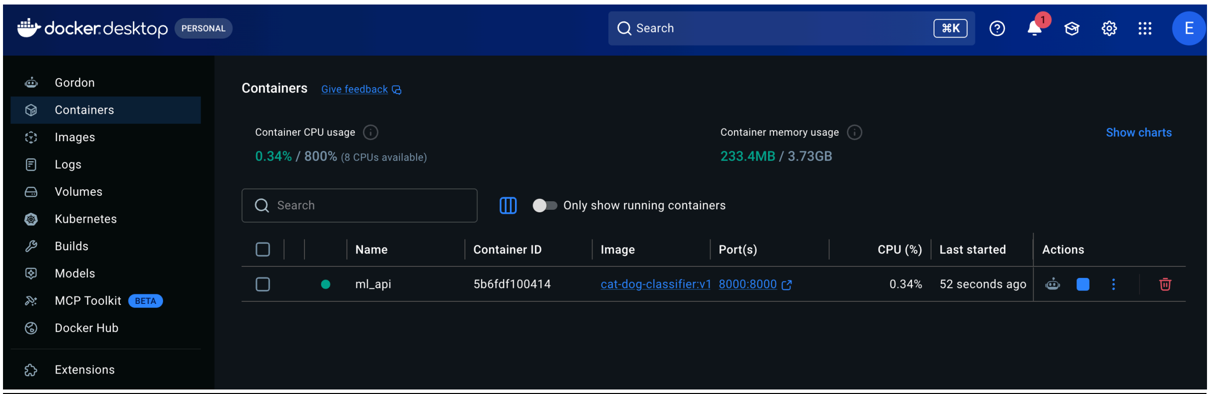
- **Endpoint Tested:** `POST /predict`
- **Sample Input:** `3.jpg`
- **Prediction Output:** `Cat` (Confidence: **99.996%**)



4. Containerization with Docker

The application was containerized using a multi-stage Docker build with a Python 3.9 base image to ensure environment portability and isolation.

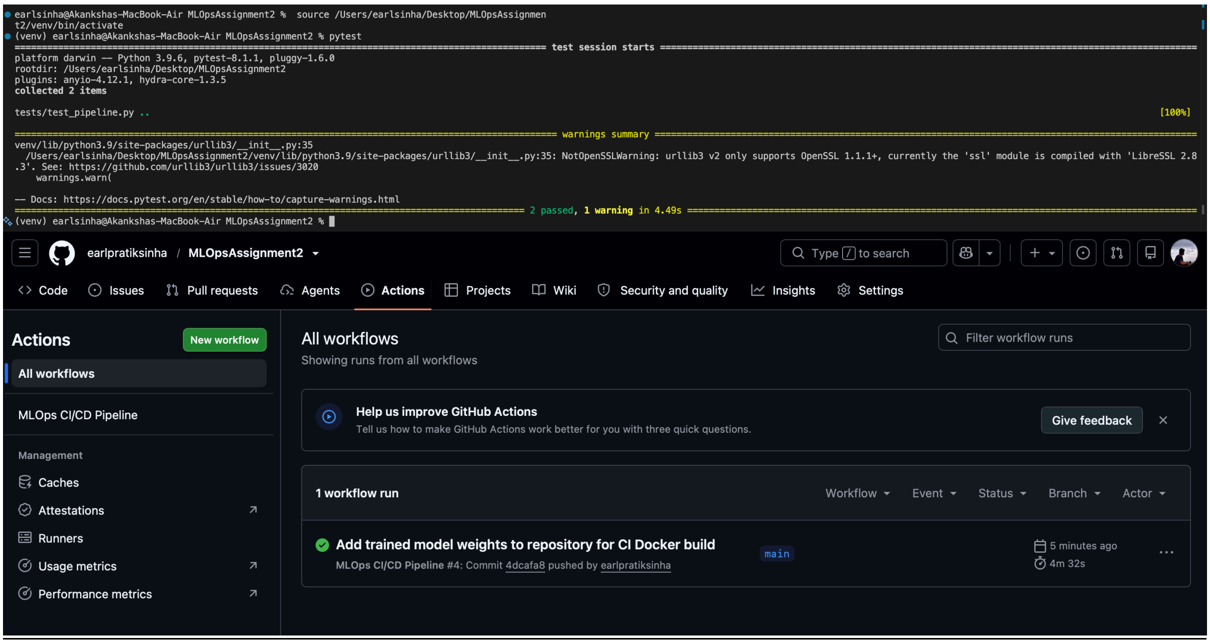
- **Container Image:** `cat-dog-classifier:v1`
- **Container Name:** `ml_api` (Container ID: `5b6fdf100414`)
- **Port Mapping:** Exposed locally on `0.0.0.0:8000->8000/tcp`

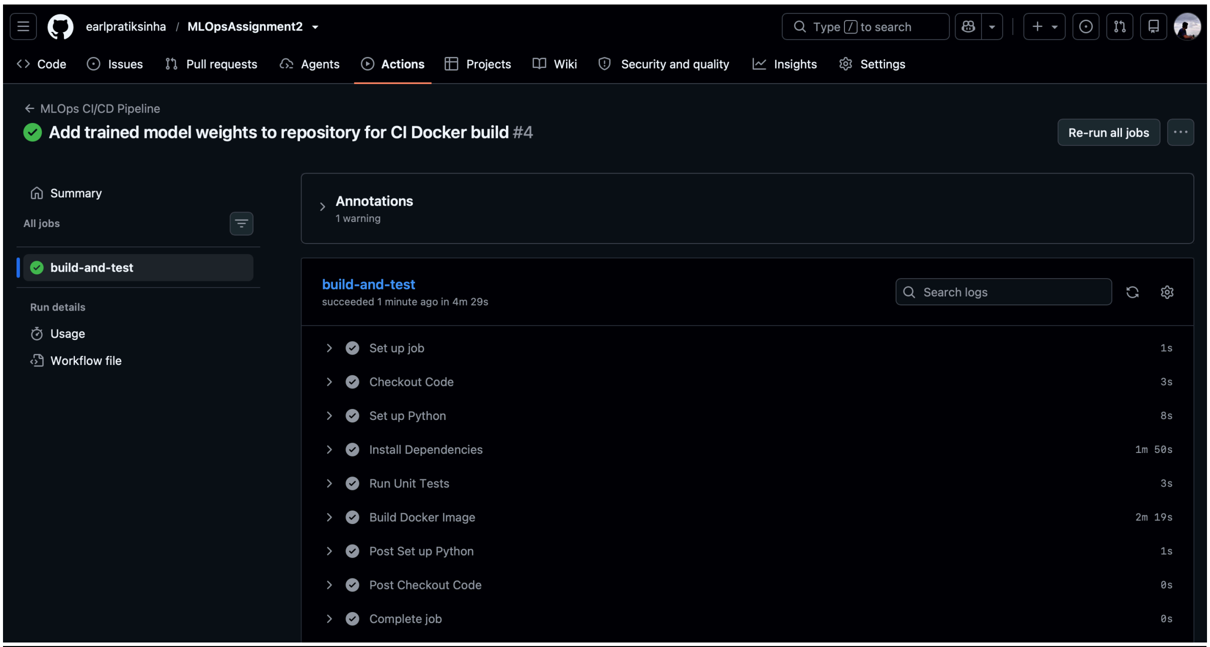


---

5. Automated Testing & Continuous Integration (CI) Unit tests were implemented using `pytest` to validate data preprocessing pipeline dimensions and image transformations. An automated CI workflow was configured via GitHub Actions to run tests and verify container builds on every push to the `main` branch.

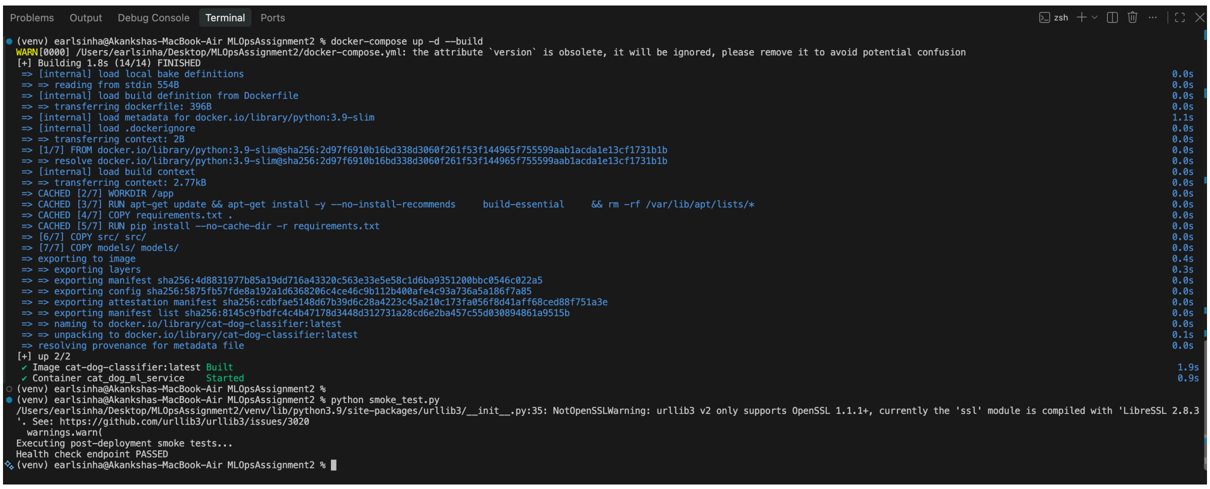
- **Unit Testing:** Executed `pytest tests/` (2 tests passed).
- **CI Workflow:** Configured `.github/workflows/ci.yml` for automated linting, testing, and Docker image builds.





6. Deployment, Health Smoke Testing & Monitoring (M4 & M5) Infrastructure deployment was configured using `docker-compose.yml` to orchestrate service delivery. The FastAPI application incorporates an explicit `/health` endpoint and structured request/latency logging. Post-deployment integrity is verified using an automated smoke testing script.

- **Deployment Manifest:** `docker-compose.yml` mapping exposed port `8000:8000`.
- **Smoke Testing:** Executed `python smoke_test.py` confirming `/health` endpoint status.
- **Logging & Monitoring:** Request metadata, prediction labels, and latency metrics logged to standard output.



Project Structure
MLOpsAssignment2/
├── src/
│ ├── app.py # FastAPI service
│ ├── train.py # Model training

```
|   └── preprocess.py   # Data preprocessing
├── tests/
|   └── test_pipeline.py
├── models/
|   └── best_model.pth
├── data/
|   ├── raw/            # Original dataset (DVC tracked)
|   └── processed/      # Preprocessed data
├── .github/workflows/
|   └── ci.yml
├── Dockerfile
├── docker-compose.yml
├── dvc.yaml
└── requirements.txt
```

Future Work
- Implement model monitoring with Prometheus/Grafana
- Add data drift detection
- Deploy to cloud (AWS/GCP) with autoscaling
- Use ONNX for faster inference